

TST Kentucky Community Platform — Project Context / Handoff

Paste this whole document into a new chat (ChatGPT or otherwise) as the first message, along with the repo/zip, so it has full context without you having to re-explain everything.

What this project is

Replacing the TST Kentucky Wix site with a small custom web app for a community of ~50-75 people. Three core features:

1. **Member-only accounts** — no public signup. People apply, an admin approves them, then they can log in.
2. **Check-in map** — members can opt in to sharing an approximate location so others can see who's nearby.
3. **Anonymous, one-person-one-vote polls** — with non-repudiation (proof someone voted) but NOT vote-attribution (nobody can see how they voted, not even admins).

Ranked-choice voting is a planned future feature, deliberately not built yet (see "Deferred / not built" below).

Hard constraints that shaped every decision

- **Budget:** aiming to stay near \$2-5/month total hosting cost.
- **~50-75 users total** — small scale, so simple/boring solutions were chosen over "enterprise" complexity everywhere possible.
- **No Vercel** (used for other frontend projects — a deliberate choice to use Cloudflare Pages instead here).
- Has an existing Fly.io account already.
- Not a technical team beyond the one developer — bias toward things that are easy to reason about and maintain solo.

Tech stack (decided, not up for debate without a good reason)

- **Frontend:** Vue 3 + Vite → deploys to **Cloudflare Pages** (free tier).
- **Backend:** Node.js + **Fastify** → one small **Fly.io** machine.
- **Database: SQLite** (via `better-sqlite3`) on a Fly.io volume — deliberately NOT Fly's "Managed Postgres" product, which starts around \$38/mo and blows the budget. SQLite + Litestream backups (to Cloudflare R2 free tier) was chosen instead.
- **Map:** Leaflet + OpenStreetMap tiles — free, no Google Maps billing.
- **Member auth:** passwordless magic-link email login (no passwords to manage or reset).
- **Admin auth:** intended to be FIDO2/WebAuthn hardware security keys (2 admins, each with a YubiKey) — **not yet implemented**, currently a placeholder shared-passcode stub. This is the most important unfinished piece.

The core design problem this project had to solve

"Non-repudiation" and "anonymous ballot" sound contradictory, and real government e-voting cryptography (blind signatures etc.) is overkill for this scale. The solution implemented:

- When a member requests to vote, the system **immediately and permanently records that member X participated** in poll Y (in a `poll_participation` table, keyed by `member_id`) — this is the non-repudiation piece, and it also enforces one-vote-per-member.
- In the same transaction, the member is issued a **random, anonymous, single-use ballot token** (in a `ballot_tokens` table) that has NO link to their identity.
- The actual vote is submitted using ONLY that token — the `votes` table has no `member_id` column at all.
- The `votes` table is **hash-chained** (each row's hash depends on the previous row's hash) so any after-the-fact tampering with results is detectable — anyone can call a `/verify` endpoint to check the chain.

This split (participation ledger vs. anonymous vote ledger) is the central design idea of the whole project. Don't undo it without understanding why it's there.

Repo layout

```

backend/
  src/
    schema.sql      - all tables, heavily commented explaining WHY
    db.js           - opens the sqlite file, applies schema on boot
    lib/tokens.js   - login tokens, ballot tokens, hash-chain logic
    lib/sessions.js - session creation/lookup/expiry (see below)
    lib/mailer.js   - email STUB (console.log, not real email yet)
    routes/auth.js  - member magic-link login
    routes/members.js - apply / admin approve-deny
    routes/checkins.js - opt-in location check-ins
    routes/polls.js - poll creation, voting, tally, verify
    routes/admin-auth.js - PLACEHOLDER, replace with WebAuthn
    server.js       - wires everything together
  .env.example
frontend/
  src/
    views/          - Home, Apply, Login, LoginVerify,
                    - CheckIn, Polls, AdminQueue
    api.js          - fetch wrapper for the backend
    router.js
  README.md

```

Auth session details (recently hardened — don't regress this)

Originally the login cookie held the raw member ID directly, which meant anyone could forge a cookie and impersonate a member. This was fixed:

- Cookie now holds a **random, signed session ID** (via `@fastify/cookie` with a `SESSION_SECRET`), never a member ID directly.
- A `sessions` table maps session ID → member ID with a real **30-day idle expiry**, checked server-side on every request via `getMemberFromSession()` in `lib/sessions.js`.
- Logging out actually deletes the session row — the old cookie stops working immediately, not just client-side.
- The server **refuses to start** if `SESSION_SECRET` is missing from `.env` (this was a real bug caught during testing — without a secret, `@fastify/cookie` silently no-ops its signing, which would silently disable auth entirely).

Deferred / not built yet — known gaps

1. **Real WebAuthn/FIDO2 admin auth.** `(routes/admin-auth.js)` is currently a single shared passcode via `(x-admin-key)` header/env var `(ADMIN_STUB_KEY)`. This needs to become real per-admin FIDO2 key registration/verification using `(@simplewebauthn/server)`. The `(admin_credentials)` table in `(schema.sql)` already has the right shape waiting for this (`credential_id`, `public_key`, `counter` columns). **This is the highest-priority remaining piece.**
2. **Real email sending.** `(lib/mailer.js)` just logs "emails" to the console. Needs a real provider (Resend, Postmark, SES, etc.) before go-live. Keep the same `(sendMail(to, subject, body))` signature so nothing else has to change.
3. **Ranked-choice voting.** Not built by design yet. `(polls.ranked_choice)` is a reserved flag/column for it. When added: the ballot payload becomes an ordered list of choices instead of a single string, and the tally function needs an instant-runoff variant. The participation/non-repudiation/anonymity design does NOT need to change for this.
4. **Wix content migration.** Old site content needs to be manually copied (text + images) into new Vue components — Wix doesn't export real code, only lets you view/copy the rendered content.
5. Location precision on the check-in map — currently uses raw browser geolocation; worth considering rounding/fuzzing for privacy.
6. Admin key loss policy is deliberately "no recovery if both keys are lost" (documented in the admin guide) — a real trade-off the org has already agreed to, not an oversight.

What's already tested and working end-to-end

Applied → admin approves → member logs in via magic link → admin opens a poll → member requests anonymous ballot → votes → double-voting is blocked → admin closes poll and sees tally → hash-chain verifies clean. Session forgery was also tested and correctly rejected.

What to ask the person before making changes

- If touching auth: confirm whether they want to tackle real WebAuthn next (it's the biggest remaining gap) or something else first.
- If touching the DB schema: note that `(backend/data/tstky.sqlite)` is generated on first run, not checked into git — don't try to "find" it in the repo, and don't worry about migrating an existing one during early development (just delete and let it regenerate).
- Don't reintroduce Vercel, Google Maps (billing risk), or Fly Managed Postgres (cost) as suggestions — all were deliberately ruled out.